



UNIVERSIDADE FEDERAL RURAL DO SEMI-ÁRIDO
CAMPUS PAU DOS FERROS
BACHARELADO EM ENGENHARIA DE COMPUTAÇÃO

**IMPLEMENTAÇÃO E AVALIAÇÃO DE UMA REDE MLP PARA DETECÇÃO
DE INTRUSÃO UTILIZANDO O DATASET NSL-KDD**

JOSÉ FERREIRA SOUSA NETO
TÚLIO GOMES DE ARAÚJO FEITOSA

Pau dos Ferros – RN

Maio de 2026

JOSÉ FERREIRA SOUSA NETO

TÚLIO GOMES DE ARAÚJO FEITOSA

**IMPLEMENTAÇÃO E AVALIAÇÃO DE UMA REDE MLP PARA DETECÇÃO DE
INTRUSÃO UTILIZANDO O DATASET NSL-KDD**

Relatório apresentado na disciplina de Sistemas Inteligentes como requisito para compor parte da nota da segunda unidade. Professor: Dr. Pedro Thiago Valério de Souza.

PAU DOS FERROS – RN

Maio de 2026

Sumário

1	Preparação dos Dados	4
2	Implementação da Rede MLP	5
3	Treinamento e Monitoramento	6
4	Avaliação Final do Modelo	7
5	Análise dos Erros em Sistemas IDS	8
6	Discussão e Conclusão	9

1 Preparação dos Dados

O conjunto de dados utilizado neste projeto foi o *NSL-KDD Dataset*, amplamente empregado em pesquisas relacionadas à detecção de intrusão em redes de computadores. O dataset representa conexões de rede classificadas como normais ou pertencentes a diferentes categorias de ataques cibernéticos.

Foram utilizados os arquivos `KDDTrain+.txt` e `KDDTest+.txt`, contendo respectivamente 125.973 amostras para treinamento e 22.544 amostras para teste. Cada amostra possui 41 atributos relacionados às características do tráfego de rede, incluindo atributos básicos da conexão, atributos de conteúdo e estatísticas de tráfego.

Inicialmente, foi realizada a identificação dos atributos categóricos presentes no conjunto de dados. Observou-se a seguinte composição:

- **protocol_type**: 3 valores únicos
- **service**: 70 valores únicos
- **flag**: 11 valores únicos

A distribuição das classes no conjunto de treino foi:

- **normal**: 67.343 amostras (53,46%)
- **ataque**: 58.630 amostras (46,54%)

Já no conjunto de teste, a distribuição observada foi:

- **normal**: 9.711 amostras (43,08%)
- **ataque**: 12.833 amostras (56,92%)

Em seguida, aplicou-se *One-Hot Encoding* aos três atributos categóricos (`protocol_type`, `service` e `flag`). Essa técnica transforma atributos categóricos em representações binárias, permitindo que a rede neural processe adequadamente informações não numéricas.

Após a codificação, o número total de atributos aumentou de 41 para 122 features:

- **Features originais**: 41
- **Features após encoding**: 122

Posteriormente, realizou-se a binarização do rótulo de saída conforme especificado na atividade:

normal $\rightarrow$ 0

qualquer ataque $\rightarrow$ 1

Após essa etapa, foi aplicada normalização utilizando padronização Z-score em todos os atributos numéricos. A normalização foi realizada utilizando exclusivamente as estatísticas do conjunto de treino, evitando vazamento de informação para o conjunto de teste.

A transformação Z-score é dada por:

$$z = \frac{x - \mu}{\sigma}$$

onde μ representa a média da feature e σ o desvio padrão.

Essa etapa é fundamental para o treinamento de redes neurais profundas, pois atributos em escalas muito diferentes podem causar instabilidade numérica, dificultando a convergência do algoritmo de otimização. Após a normalização, verificou-se que as médias ficaram próximas de zero e os desvios padrão próximos de um, indicando pré-processamento correto.

Ao final da preparação dos dados, obteve-se:

- **X_train:** (125973, 122)
- **X_test:** (22544, 122)

2 Implementação da Rede MLP

A arquitetura escolhida para o problema foi uma Rede Perceptron Multicamadas (*Multilayer Perceptron – MLP*) implementada utilizando a biblioteca PyTorch.

O modelo foi construído herdando da classe `nn.Module`, conforme recomendado pela própria framework. A função de perda utilizada foi a `BCEWithLogitsLoss`, apropriada para problemas de classificação binária.

A escolha dessa função de perda é adequada porque ela combina internamente a função sigmoide com a entropia cruzada binária, proporcionando maior estabilidade numérica durante o treinamento.

Os dados foram encapsulados utilizando `TensorDataset` e carregados através de `DataLoader` com mini-batches de tamanho 256.

Além disso, reservou-se 20% do conjunto de treino para validação, permitindo monitoramento da capacidade de generalização da rede durante o treinamento.

A topologia escolhida como melhor configuração foi denominada *medium*, composta pelas seguintes camadas ocultas:

[128, 64, 32]

Também foi utilizado *dropout* com taxa de 0,3 para auxiliar na redução de overfitting.

A arquitetura final apresentou:

- **Número de camadas ocultas:** 3
- **Neurônios:** 128, 64 e 32
- **Dropout:** 0,3
- **Total de parâmetros:** 26.561

3 Treinamento e Monitoramento

O treinamento da MLP foi realizado durante 50 épocas utilizando o otimizador Adam com taxa de aprendizagem:

$$\eta = 0,001$$

Também foi implementada estratégia de *Early Stopping* com paciência igual a 10 épocas, visando evitar sobreajuste e interromper o treinamento caso a perda de validação deixasse de melhorar.

Durante o treinamento, foram monitoradas as perdas de treino e validação. Os resultados demonstraram comportamento consistente de convergência ao longo das épocas.

A Tabela 1 apresenta alguns valores registrados durante o treinamento.

Época	Train Loss	Validation Loss
5	0,025339	0,020882
10	0,020611	0,019015
15	0,018106	0,015530
20	0,016279	0,013944
25	0,015155	0,012784
30	0,014186	0,011899
35	0,013091	0,013341
40	0,011922	0,011880
45	0,012527	0,010815
50	0,012071	0,010035

Tabela 1: Evolução das perdas de treino e validação

O melhor valor de perda de validação obtido foi:

$$\text{Validation Loss}_{\min} = 0,009611$$

As curvas de treinamento e validação foram armazenadas na figura `loss_curves.png`.

Os resultados demonstram que a rede conseguiu aprender padrões relevantes do tráfego de rede sem apresentar divergência ou instabilidade significativa durante o treinamento.

4 Avaliação Final do Modelo

Após o treinamento, o modelo foi avaliado utilizando o conjunto de teste `KDDTest+.txt`. Para transformar as saídas contínuas da rede em rótulos binários, aplicou-se a função sigmoide seguida de limiar de decisão igual a 0,5.

As métricas obtidas foram:

Acurácia = 80,86%

Precisão = 97,24%

Recall = 68,32%

F1-score = 80,25%

Os resultados indicam que o modelo apresentou elevada precisão na identificação de ataques, embora ainda existam falsos negativos relevantes.

A matriz de confusão obtida é apresentada na Tabela 2.

		Classe Real	
		Normal	Ataque
Previsto	Normal	9462	4066
	Ataque	249	8767

Tabela 2: Matriz de confusão da MLP

A análise detalhada mostrou:

- **True Negatives:** 9462
- **True Positives:** 8767
- **False Positives:** 249
- **False Negatives:** 4066

Observa-se que o número de falsos positivos foi relativamente baixo, indicando que o sistema raramente classificou conexões legítimas como ataques. Entretanto, o número de falsos negativos ainda foi significativo.

5 Análise dos Erros em Sistemas IDS

Em Sistemas de Detecção de Intrusão (*Intrusion Detection Systems – IDS*), os falsos negativos representam o erro mais crítico.

Um falso negativo ocorre quando um ataque é classificado incorretamente como conexão normal. Nesse cenário, a invasão permanece não detectada, permitindo que o atacante continue explorando vulnerabilidades do sistema sem gerar alertas.

Por outro lado, falsos positivos correspondem a conexões legítimas classificadas como ataques. Embora isso gere alarmes desnecessários e aumente a carga operacional da equipe de segurança, o sistema ainda permanece protegido contra ameaças reais.

Portanto, em aplicações reais de segurança cibernética, é preferível aceitar um número maior de falsos positivos caso isso reduza significativamente os falsos negativos.

Essa característica influencia diretamente a escolha do limiar de decisão da rede neural. Limiar mais baixos aumentam o recall do sistema, reduzindo falsos negativos, enquanto limiares mais altos priorizam precisão e reduzem falsos positivos.

A Tabela 3 apresenta os resultados obtidos para diferentes limiares de decisão.

Limiar	Acurácia	Precisão	Recall	F1	FP	FN
0,1	0,8263	0,9674	0,7191	0,8250	311	3605
0,2	0,8177	0,9695	0,7018	0,8142	283	3827
0,3	0,8127	0,9711	0,6915	0,8078	264	3959
0,4	0,8108	0,9718	0,6875	0,8053	256	4010
0,5	0,8086	0,9724	0,6832	0,8025	249	4066
0,6	0,8073	0,9732	0,6801	0,8007	240	4105
0,7	0,8049	0,9731	0,6760	0,7978	240	4158
0,8	0,8025	0,9731	0,6716	0,7947	238	4214
0,9	0,7989	0,9731	0,6652	0,7902	236	4297

Tabela 3: Análise de diferentes limiares de decisão

Observa-se que limiares menores aumentam a capacidade de detecção de ataques, reduzindo o número de falsos negativos. Dessa forma, para aplicações reais de IDS, limiares entre 0,3 e 0,4 podem representar escolha mais adequada.

6 Discussão e Conclusão

Os resultados obtidos demonstram que a Rede MLP apresentou desempenho satisfatório para o problema de detecção de intrusão utilizando o dataset NSL-KDD.

O pré-processamento dos dados mostrou-se fundamental para o sucesso do treinamento, especialmente a aplicação de One-Hot Encoding e normalização Z-score. A expansão das variáveis categóricas permitiu que a rede neural representasse adequadamente protocolos, serviços e estados de conexão.

A arquitetura multicamadas conseguiu capturar relações não lineares presentes no tráfego de rede, superando limitações que modelos lineares apresentariam nesse tipo de problema.

O treinamento apresentou convergência estável ao longo das épocas, atingindo baixa perda de validação sem evidências severas de overfitting.

Em relação às métricas finais, a rede apresentou precisão bastante elevada, indicando forte capacidade de distinguir conexões legítimas de atividades maliciosas. Entretanto, o recall ainda pode ser melhorado, já que parte dos ataques permaneceu não detectada.

Em aplicações reais de segurança computacional, falsos negativos representam risco significativamente maior do que falsos positivos, pois permitem que invasões reais ocorram sem detecção. Assim, a definição do limiar de decisão deve priorizar aumento do recall mesmo ao custo de maior quantidade de alarmes falsos.

De maneira geral, conclui-se que a MLP mostrou-se adequada para o problema proposto, apresentando boa capacidade de generalização e desempenho consistente para classificação binária de tráfego de rede em sistemas IDS.